Task 10 — Final Practical Challenge

DEVOPS-007: Dockerfile & Custom Docker Image | Time: 1 Hour

Requirement: Build and containerize a small company website using Nginx. Create a custom Dockerfile, build a versioned Docker image, run it as a container, expose it through a host port, troubleshoot one intentionally introduced error, and push the final image to Docker Hub.

Step 1 — Create the Application

Created a simple static website folder for the company site.

Project structure

```
company-website/
  ■■■ index.html
  ■■■ style.css
  ■■■ Dockerfile
  ■■■ .dockerignore
```

index.html

```
<!DOCTYPE html>
<html>
<head><title>Acme Co</title><link rel="stylesheet" href="style.css"></head>
<body>
  <h1>Welcome to Acme Co</h1>
  <p>Your trusted partner in DevOps solutions.</p>
</body>
</html>
```

Step 2 — Create the Dockerfile

```
FROM nginx:alpine
WORKDIR /usr/share/nginx/html
COPY index.html .
COPY style.css .
EXPOSE 80
```

Used nginx:alpine as the base image since it's lightweight and comes with Nginx preconfigured to serve files from /usr/share/nginx/html. WORKDIR sets that as the target folder, and COPY places the site files there. EXPOSE 80 documents that the container listens on port 80, Nginx's default.

Step 3 — Create .dockerignore

```
.git
*.md
node_modules
.DS_Store
```

Prevents unnecessary files from being copied into the build context, keeping the image clean and the build faster.

Step 4 — Build the versioned Docker image

```
docker build -t acme-website:v1 .
```

Builds the image from the Dockerfile in the current directory and tags it acme-website:v1, giving it a clear, traceable version instead of relying on latest.

Step 5 — Run the container with port mapping

```
docker run -d --name acme-web -p 8080:80 acme-website:v1
```

Runs the image in detached mode, names the container acme-web, and maps host port 8080 to the container's port 80 so the site is reachable at localhost:8080.

Step 6 — Test the website

```
curl localhost:8080
```

Confirmed the HTML content was returned successfully — the "Welcome to Acme Co" page loaded, verifying the container was serving the site correctly on the mapped port.

Step 7 — Troubleshoot an Intentionally Introduced Error

Introduced error

```
Changed the COPY instruction to reference the wrong filename:
COPY indx.html .
```

Rebuild attempt

```
docker build -t acme-website:v1 .
```

Error observed

```
ERROR: failed to calculate checksum of ref: "/indx.html": not found
```

Investigation

Checked the project folder contents and confirmed the file was actually named index.html — the Dockerfile had a typo (missing the "e").

Root cause

A filename typo in the COPY instruction meant Docker's build context couldn't locate the file to copy.

Fix

```
Corrected the line back to: COPY index.html .
```

Verification

Rebuilt with docker build -t acme-website:v1 . — the build completed without errors, and re-running curl localhost:8080 (after restarting the container) confirmed the site loaded correctly again.

Step 8 — Create a v2 image

Updated the site content (added a new paragraph to index.html), then built a second, distinct version:

```
docker build -t acme-website:v2 .
```

Step 9 — Push the Final Image to Docker Hub

```
docker login
docker tag acme-website:v2 mydockerhubuser/acme-website:v2
docker push mydockerhubuser/acme-website:v2
```

docker login authenticates with Docker Hub. docker tag renames the local image to match the <dockerhub-username>/<repo>:<tag> format Docker Hub requires. docker push then uploads that image to the remote repository, making it available for anyone with access to pull.

Step 10 — Pull and Run Again (verification)

```
docker pull mydockerhubuser/acme-website:v2
docker run -d --name acme-web-v2 -p 8081:80 mydockerhubuser/acme-website:v2
```

Pulling the image back down from Docker Hub and running it confirms the image was pushed successfully and works identically outside the original build environment — proving the image is portable.

Workflow Summary

Stage	Command / Action
Create Application	Static site: index.html, style.css
Create Dockerfile	FROM nginx:alpine + COPY + EXPOSE 80
Create .dockerignore	Excludes .git, *.md, etc.
docker build	docker build -t acme-website:v1 .
docker run + Port Mapping	docker run -d -p 8080:80 --name acme-web acme-website:v1
Test Website	curl localhost:8080
Troubleshoot	Fixed COPY typo (indx.html → index.html)
Create v2	docker build -t acme-website:v2 .
Docker Hub	docker login + docker tag
docker push	docker push mydockerhubuser/acme-website:v2
docker pull / Run Again	Verified image works when pulled fresh

Note: mydockerhubuser is a placeholder for the trainee's real Docker Hub username. docker login and docker push require an actual Docker Hub account and cannot be performed without live credentials.